HarnessSpec: A Protocol for an Orchestrator LLM to Design, Validate and Revise a Subagent's Harness

Syed Affan Daimi · M.S. CS, IIIT Hyderabad · CS6.403 AutoSE, Monsoon 2026 · Project proposal (registered report) · 21 Oct 2026

1. Problem statement and motivation

When a coding agent delegates work, someone must decide the subagent's **harness**: its instructions, the slice of repository context it sees, the tools it may call, its step and token budgets, when it must stop, how its result is checked and what it must hand back. Today this is hand-written once per system, or left to an orchestrator's free-text brief. Anyone running agents on local models feels the consequences. Harness choice changes tokens per solved task by up to 40× while moving pass rate by only 0–8 points [1], so harness design is mainly a cost and energy lever. And delegation without a checkable return hides failures: in a repository question-answering study, 41.8% of a delegated architecture's failures occurred at the planner-to-subagent handoff, usually silently with a confident wrong answer, and the delegated design scored 46.2% against 65.2% for plain retrieval at more than twice the cost per correct answer [2]. Harness design is thus a first-order lever for cost, energy and the reliability of delegation, yet it is made informally and without checks.

2. Existing approaches, limitations and opportunities

Offline search: ADAS lets a meta agent program new agents guided by an archive [3]; Meta-Harness uses an agentic proposer that reads traces of earlier harness candidates (+7.7 points over a context-management baseline with 4× fewer context tokens on text classification) [4]; ReCreate has an optimizer agent turn execution evidence into scaffold edits [5]. All tune a harness for a task family over many costly runs. **Online:** Live-SWE-agent evolves its own scaffold at runtime (77.4% on SWE-bench Verified) [6]; AOrchestra, the closest work, has an orchestrator fill an (Instruction, Context, Tools, Model) tuple for each on-demand subagent [7]; ToolOrchestra trains an 8B orchestrator to route among models and tools [8]. **Practice:** a source study of eleven coding harnesses finds behavioural policy moving from prompt prose into configuration [9]. **Opportunities.** (i) As far as AOrchestra's method description shows, the generated tuple is not validated, subagents get no per-agent budget or turn cap, and results return as summaries, which is where the handoff failures in [2] arise. (ii) The cost of the design call itself is rarely charged against the savings, so net benefit is unknown. (iii) Its reported results use Gemini-3-Flash; small local orchestrators and subagents are untested.

3. Objectives and scope

- **O1 – Define HarnessSpec**, a typed protocol (JSON schema) with eight fields: goal and acceptance tests; context slice; tool allowlist with permission levels; step, token and time budgets; stop and escalation rules; verification hooks (tests, linters, AST checks); a *return contract* that demands evidence (file and line citations, test output); and model tier.
- **O2 – Build the compiler and runtime:** a deterministic validator (schema, tools exist, no tool permission broader than the parent's, budgets feasible) that returns typed errors for the orchestrator to fix within a bounded number of rounds; a runtime that enforces budgets, tool guards, hooks and the return contract and logs traces; and trace-driven revision.
- **O3 – Compare four designers at equal total budget:** (A) fixed generic harness; (B) free-text brief; (C) four-field tuple, AOrchestra-style (own re-implementation); (D) full HarnessSpec with validator and revision. Tasks: ~60 SWE-bench Verified issues and ~60 SWE-QA repository questions. Orchestrator and subagent are 7–14B or 24–30B open-weight models served by Ollama.

Out of scope: training or fine-tuning the orchestrator; offline population search as in ADAS or Meta-Harness (we test per-task design plus a small spec cache); parallel multi-agent coordination; security guarantees beyond least-privilege allowlists; non-coding domains.

4. Proposed direction

Orchestrator LLM reads task, repo summary and a subagent capability card; emits a HarnessSpec	**Validator / compiler** deterministic checks; typed errors; ≤ 3 repair rounds	**Subagent runtime** edit–test loop under enforced budgets, tool guards, hooks	**Return gate** result must carry evidence; unsupported claims bounce	**Revise or reuse** trace-driven edits; good specs cached by task type

The orchestrator inspects the task and repository, then writes the spec; the validator rejects anything malformed, over-privileged or unaffordable; the subagent runs inside the compiled harness; the return gate refuses results that lack the evidence the spec demanded; and the trace feeds either a revised spec or an escalation to the orchestrator. **Why agentic:** choosing a harness means reasoning about this task, this repository and this subagent's limits, which cannot be enumerated as a fixed configuration; the subagent works in an open-ended tool loop; and failures must be diagnosed from traces and acted on. Verification stays *deterministic* and outside the LLMs, so it is cheap, auditable and not self-reported.

5. Success and quality attributes

Pre-registered hypotheses. H1: D is non-inferior to A on accuracy (margin 5 points) with fewer total tokens, counting the design call. **H2:** D has fewer silent handoff failures (confident but unsupported returns) than B and C. **H3:** validation plus revision (D vs C) raises the first-pass valid-spec rate, most for 7–14B orchestrators. **H4:** cached specs amortise the design call; we report the break-even number of tasks. Negative results will be reported as such.

Metrics. Primary: issue-resolution rate and answer accuracy; total tokens per correct result (design, subagent and revisions); silent-handoff rate. Secondary: first-pass spec validity, revision rounds, budget violations intercepted, wall-clock time, and energy from CPU (RAPL) and GPU (NVML) counters as success per kWh [10]. **Rigour:** ≥3 seeds per condition, paired bootstrap 95% CIs, pinned model digests and containers, published traces. **Quality attributes:** validator determinism and auditability (same spec, same verdict); least privilege; reproducibility; sustainability (local models only, energy reported beside every accuracy figure). **Risks:** small orchestrators may write poor specs (measured as validity rate) and design overhead may exceed savings (H4). Substantial AI-tool use will be logged in the repository (tools, prompts, outputs, verification).

[1] The Scaffold Effect in Coding Agents, arXiv:2607.22585. [2] Deep Agentic Search for Repository-Level Code Question Answering: An Empirical Study, arXiv:2608.01507. [3] Automated Design of Agentic Systems (ADAS), arXiv:2408.08435. [4] Meta-Harness: End-to-End Optimization of Model Harnesses, arXiv:2603.28052. [5] ReCreate: Reasoning and Creating Domain Agents Driven by Experience, arXiv:2601.11100. [6] Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly?, arXiv:2511.13646. [7] AOrchestra: Automating Sub-Agent Creation for Agentic Orchestration, arXiv:2602.03786. [8] ToolOrchestra: Elevating Intelligence via Efficient Model and Tool Orchestration, arXiv:2511.21689. [9] Harness Engineering: Anatomy, Architecture, and Evolution of Coding Agents, arXiv:2609.00006. [10] SWEnergy: An Empirical Study on Energy Efficiency in Agentic Issue Resolution Frameworks with SLMs, arXiv:2512.09543.